PROJECT AND TEAM INFORMATION

Project Title

ArenaForge: Esports Tournament, Ranking, and Smart Matchmaking System

Student / Team Information

Team Name: *Team ID: DSCPP-III-2026-T104*	*GameDev*
Team member 1 (Team Lead) *(Bhatt, Nimisha: 2510018086: 2510018086@geu.ac.in):*	*Bhatt, Nimisha – 2510018086 2510018086@geu.ac.in* 
Team member 2 *(Rawat, Kushagra: 2510370072: 2510370072@geu.ac.in):*	*Rawat, Kushagra – 2510370072 2510370072@geu.ac.in* 
Team member 3 *(Kandwal, Manashvi: 2510380299: 2510380299@geu.ac.in):*	*Kandwal, Manashvi – 2510380299 2510380299@geu.ac.in* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

- Esports tournaments are becoming increasingly popular among students, gaming communities, and competitive players.
- Managing tournaments manually becomes difficult when the number of players and teams increases.
- Tournament organizers need to manage:
  - Player registration
  - Team formation
  - Rankings
  - Matchups
  - Schedules
  - Results
  - Leaderboards
- Manual tournament management can result in:
  - Unfair matchups
  - Repeated opponents
  - Scheduling conflicts
  - Incorrect rankings
  - Difficulty maintaining match records
- There is a need for a simplified system that demonstrates how DSA can be applied to real-world tournament management.
- ArenaForge will provide a centralized system for managing esports tournaments.
- Players will be able to:
  - Register
  - Form teams
- Administrators will be able to:
  - Create tournaments
  - Manage participants
- Teams will be ranked according to their performance and seeded accordingly.
- Tournament brackets will be generated automatically according to the selected tournament format.
- A smart matchmaking system will pair teams with similar rankings.
- The matchmaking system will also consider previous opponents to reduce repeated matchups.
- Scheduling conflicts between teams will be detected using graph-based techniques.
- Match results will be recorded and used to update rankings and leaderboards.

- Different data structures will be used for specific tournament-management tasks:
  - Sorting algorithms will be used for ranking and seeding.
  - Heaps and priority queues will be used for matchmaking.
  - Trees will be used to represent tournament brackets.
  - Hash tables or Binary Search Trees will provide efficient player and team searching.
  - Queues will manage player registration and waiting lists.
  - Linked lists will maintain match history.
  - Graphs will help identify scheduling conflicts.
- C++ OOP concepts will be used to structure the application.
- The project will provide practical understanding of:
  - Data Structures and Algorithms in C
  - Object-Oriented Programming in C++
- The main aim is to develop an organized, efficient, and fair esports tournament management system.

State of the Art / Current solution (1 pt)

- Esports tournaments are currently managed using dedicated platforms such as Challonge and Battlefy.
- These platforms provide features such as:
  - Player registration
  - Team management
  - Tournament creation
  - Tournament brackets
  - Match scheduling
  - Rankings
  - Result tracking
  - Leaderboards
- Some existing systems also provide ranking-based matchmaking.
- Commercial platforms are designed mainly as ready-to-use tournament management services.
- The internal implementation of their algorithms and data structures is generally not the primary focus for users.
- Understanding the underlying algorithms is important from an educational and development perspective.
- ArenaForge will implement the core tournament-management operations from scratch using C and C++.
- Sorting algorithms will be used for ranking and participant seeding.
- Priority queues and heaps will be used for matchmaking.
- Trees will be used for tournament bracket representation.
- Hash tables or Binary Search Trees will be used for fast player and team searching.
- Queues will be used for registration and waitlist management.
- Linked lists will be used to maintain match history.
- Graphs will be used to detect scheduling conflicts.
- C++ classes will provide an organized object-oriented application structure.
- Inheritance and polymorphism will allow different tournament formats to be implemented.
- The proposed system will therefore combine a practical esports application with direct implementation of DSA and OOP concepts.

Project Goals and Milestones (2 pts)

Project Goals

- Develop a tournament management system for registering players, creating teams, and managing tournaments.
- Implement ranking and seeding using sorting algorithms.
- Develop fair matchmaking using priority queues/heaps.
- Represent knockout tournament brackets using trees.
- Implement fast player/team searching using hash tables.
- Maintain registration queues and match history using queues and linked lists.
- Use graphs to identify scheduling conflicts between teams.
- Apply C++ OOP concepts such as inheritance, encapsulation, abstraction, and polymorphism.
- Store and retrieve player, team, tournament, and match data using files.
- To implement inheritance using the User → Player/Admin relationship.
- To implement polymorphism for different tournament formats.
- To implement exception handling for invalid operations.
- To use file handling for storing and retrieving tournament data.

Initial Milestones

1. Requirement analysis Design - Complete the features, data structure, class and architecture of the system.
2. Basic implementation using C++ – Make Player, Team, Match, Tournament, Admin and Game classes.
3. DSA Module – Implement sorting, queues, linked lists, hash tables, trees, and heaps.
4. Tournament System – Implement registration, ranking, seeding, bracket generation, and result handling.
5. Smart Matchmaking and Scheduling – Implement fairness-based matchmaking and conflict-free scheduling.
6. File Handling and Integration – Connect the C DSA engine with the C++ application and add data persistence.
7. Testing and Finalization – Test different tournament scenarios, fix errors, and prepare the final demonstration.
8. Requirement Analysis and System Design
9. Identify functional requirements.
10. Identify major system modules.
11. Identify required DSA concepts.
12. Design C++ classes and their relationships.

Project Approach (3 pts)

1. **System Design and Planning**
   - Specify all the key entities of the tournament system.
   - Determine the relationships among players, teams, matches, and tournaments.
   - Split the task in two parts – a C++ application layer and a C DSA engine.
   - Work out the system architecture and workflow.
   - Choose suitable data structures for each system operation.
2. **C++ OOP Application Development**
   - Develop the application layer using C++.
   - Create classes for:
     - User
     - Player
     - Admin
     - Team
     - Game
     - Match
     - Tournament
     - Leaderboard
   - Employ encapsulation to keep class data safe.
   - Employ inheritance to create classes for Player and Admin from User class.
   - Use abstraction for common tournament operations.
   - Use polymorphism to implement different tournament formats.
3. **DSA Engine Development in C**
   - Implement sorting algorithms for ranking and seeding.
   - Implement a heap/priority queue for matchmaking.
   - Implement trees for tournament brackets.
   - Implement hash tables or BSTs for searching.
   - Implement queues for registration and waitlists.
   - Implement linked lists for match history.
   - Implement graphs for scheduling conflict detection.
4. **Ranking and Seeding**
   - Store player/team performance information.
   - Calculate ranking scores.
   - Sort teams according to their rankings.
   - Generate tournament seeds based on ranking.
   - Update rankings after every completed match.

Smart Matchmaking

- Identify eligible teams for the next match.
- Compare team rankings.
- Calculate ranking differences.
- Check previous opponents.
- Check team availability.
- Check scheduling conflicts.
- Calculate the Fair Matchmaking Score.
- Use a priority queue/heap to prioritize suitable matchups.
- Select the most suitable available matchup.

Tournament Bracket and Match Management

- Generate the tournament bracket according to the selected format.
- Use a tree structure for knockout tournaments.
- Schedule matches.
- Record match results.
- Advance winners to the next round.
- Update rankings and leaderboards.

File Handling and Data Storage

- Store player and team information using files.
- Store tournament details.
- Store match schedules.
- Store match results.
- Store match history.
- Load existing information when the application starts.

Testing and Validation

- Test different numbers of teams.
- Test different tournament formats.
- Test ranking and seeding.
- Test fair matchmaking.
- Test repeated-opponent prevention.
- Test scheduling conflicts.
- Test invalid registrations.

System Architecture (High Level Diagram) (2 pts)

1 ArenaForge

Esports tournaments are becoming increasingly popular among students, gaming communities, and competitive players.

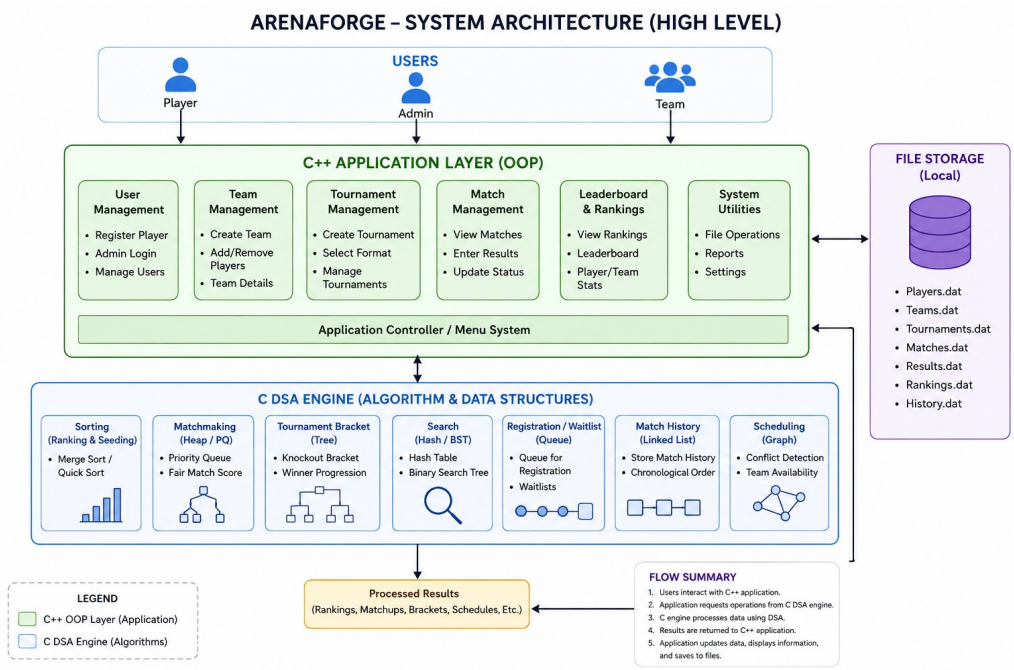


Figure 1: ArenaForge Tournament Management System

- The system consists of a C++ application layer and a C-based DSA engine.
- The C++ layer manages players, teams, tournaments, matches, and administrators, while the C module performs ranking, matchmaking, searching, bracket generation, registration, match history, and scheduling operations.
- Both components communicate to process tournament data, and file storage is used to preserve information between sessions.

Project Outcome / Deliverables (1 pt)

The main outcome of the project will be a working **ArenaForge** console application capable of managing the basic lifecycle of an esports tournament.
The expected deliverables are:

- Player and team registration and management.
- Tournament creation and participant management.
- Ranking and seeding using sorting algorithms.
- Fair matchmaking using a priority queue/heap.
- Knockout tournament bracket generation using trees.
- Player/team search using a hash table.
- Registration/waitlist management using queues.
- Match history using linked lists.
- Scheduling conflict detection using graphs.
- Leaderboard and result management.
- File-based data storage and retrieval.
- C++ classes demonstrating inheritance, encapsulation, abstraction, and polymorphism.

The final project will also include source code, test cases, sample tournament data, and documentation explaining the algorithms and OOP concepts used.

Assumptions

- The system is assumed to be used by a tournament organizer or administrator in a controlled environment.
- Players and teams are assumed to provide valid information during registration.
- Each team has a unique name and a fixed number of players according to the selected game.
- Match results are entered by the administrator and are assumed to be correct.
- The initial version will work offline using local files rather than a live online database.
- Internet connectivity and real-time game-server integration are outside the initial project scope.

References

[1] Gentile, Dina. "Overview of Esports Management." *Routledge Handbook of Esports* (2024). Book Chapter.

[2] Pizzo, A. D., Jones, G. J., Baker, B. J., Funk, D. C., and Kunkel, T. "Sensemaking of Novelty: The Dynamic Nature of Integrating Esports Within a Traditional Sport Organization" (2022). *Sport management review, 25(3), 383-405.* Journal Paper.

[3] Flegr, S., and Schmidt, S. L. "Strategic Management in eSports – A Systematic Review of the Literature" (2022). *Sport Management Review, 25(4), 631-655.* Journal Paper.

[4] GeeksforGeeks. "Data Structures and Algorithms in C/C++." Online Educational Resource. https://www.geeksforgeeks.org/

[5] Challonge. "Online Tournament Management Platform." Online Platform. https://challonge.com/

[6] Battlefy. "Esports Tournament and Competition Platform." Online Platform. https://battlefy.com/